

# A Feasibility Study on Context-Aware Process-Level Behavior Prediction for Proactive Application Preloading

Jingyi Guo, Kewei Chen, Chengyu Fan, Mosha Huang

April 14th, 2026

## 1 Abstract

This project investigates the feasibility of a cross-platform system for predicting user behavior at the process level and proactively optimizing application execution through preloading strategies. The system targets work-oriented environments on personal laptops and leverages system tracing tools, including eBPF (Linux), ETW (Windows), and dtrace (macOS), to collect application-level process data.

Focusing on frequently used productivity and development applications such as integrated development environments, document editors, browsers, and communication tools, the system models user workflows as temporal sequences of application-level process events. Based on these sequences, predictive models are developed to estimate the next likely application or process that the user will invoke.

The study explores a progressive modeling framework ranging from statistical transition models to sequence learning architectures and context-aware semantic models. These models are evaluated in terms of prediction accuracy, computational overhead, and their effectiveness in reducing application launch latency via user-level preloading mechanisms.

The proposed system is initially designed for a single-user setting with a well-defined behavioral profile and is subsequently extended toward multi-user generalization through shared representations and user-specific adaptation. The feasibility analysis addresses challenges including cross-platform inconsistency, process ambiguity, real-time constraints, and privacy considerations.

## 2 Introduction

Modern personal computing environments are characterized by increasingly complex user workflows involving frequent transitions between applications such as code editors, terminals, document processors, browsers, and communication platforms. Despite advances in hardware performance, application launch latency and inefficient resource utilization remain persistent issues, particularly in multitasking and development-heavy scenarios.

This project proposes a context-aware system that predicts user behavior at the application process level and proactively optimizes system performance through application preloading. Unlike traditional usage prediction approaches that rely on coarse-grained statistics or isolated application usage patterns, this work models user activity as structured workflows composed of sequences of interdependent process events.

The system operates at the user level and is designed to be cross-platform, supporting Linux, Windows, and macOS through their respective tracing mechanisms. By filtering out system-level processes and focusing on application-level activities, the proposed approach aims to capture meaningful patterns in user workflows, such as transitions between coding, compiling, browsing, and communication tasks.

A key objective of this study is to evaluate the feasibility of predicting the next useful application based on recent process context and user profile information. Such predictions enable proactive strategies, including background application initialization and cache warming, which can reduce perceived latency and improve user productivity.

The project adopts a staged design strategy, beginning with simple statistical models and progressing toward more sophisticated sequence learning and context-aware approaches. This progression allows for a systematic evaluation of trade-offs between model complexity, prediction accuracy, and system overhead.

Furthermore, while the initial implementation focuses on a single user with a well-understood behavioral profile, the study also investigates pathways toward generalization across users through shared models and personalization techniques.

The remainder of this report presents the system design, modeling approaches, feasibility analysis, and evaluation framework for the proposed approach.

## 3 System Design

### 3.1 Overview

The proposed system is a cross-platform, user-level framework designed to collect application-level process data, model user workflows, and predict future application usage for proactive system optimization. The system operates on personal laptops and supports multiple operating systems, including Linux, Windows, and macOS.

The architecture consists of five main components:

1. Data Collection Layer
2. Process Filtering and Abstraction Layer
3. Workflow Construction Layer
4. Prediction Model
5. Action Layer (Preloading Mechanism)

Each component is designed to balance observability, computational efficiency, and cross-platform compatibility.

### 3.2 Data Collection Layer

The system collects process-level execution data using platform-specific tracing tools:

- Linux: eBPF (extended Berkeley Packet Filter)
- Windows: ETW (Event Tracing for Windows)
- macOS: dtrace

These tools enable the capture of runtime information about process creation and execution without modifying the operating system kernel. The collected data includes process identifiers, parent-child relationships, execution timestamps, and resource usage statistics.

Formally, each recorded event is represented as:

$$e_t = (t, \text{PID}, \text{PPID}, a_t, r_t, m_t)$$

where:

- $t$  denotes the timestamp,
- PID and PPID denote process identifiers,
- $a_t$  denotes the application-level process name,
- $r_t$  denotes CPU usage or execution intensity,
- $m_t$  denotes memory usage.

### 3.3 Process Filtering and Abstraction

Raw process traces contain a large number of system-level processes that are not relevant to user workflows. Therefore, a filtering step is introduced to isolate application-level processes.

The filtering process consists of:

- Removing system daemons and background services,
- Grouping subprocesses under a unified application identity,
- Mapping low-level process names to high-level application categories.

For example, multiple subprocesses generated by a browser are mapped to a single application entity such as **Chrome** or **Safari**. Similarly, helper processes of development environments are grouped under **VS Code**.

This abstraction step is critical to reduce noise and ensure that the model captures meaningful user behavior rather than system-level artifacts.

### 3.4 Workflow Definition

User behavior is modeled as a sequence of application-level process events, referred to as a *workflow*.

$$W = (a_1, a_2, \dots, a_T)$$

where each  $a_t$  represents an application-level process observed at time  $t$ .

To incorporate temporal locality, we define a context window of length  $k$ :

$$C_t = (a_{t-k}, \dots, a_t)$$

The prediction task is then defined as:

$$P(a_{t+1} \mid C_t)$$

which represents the probability distribution over the next application given the recent workflow context.

### 3.5 Feature Representation

In addition to application identity, the system may incorporate auxiliary features to enrich the workflow representation:

- Time-based features (e.g., hour of day, session boundaries),
- Resource usage statistics (CPU, memory),
- Event duration or activity intensity,

- Optional semantic tags (e.g., compile, edit, browse, communicate).

These features can be encoded as vectors and concatenated with application embeddings for input into the prediction model.

### 3.6 Prediction Model Interface

The prediction component receives a sequence of encoded workflow states and outputs a probability distribution over candidate applications:

$$\hat{y}_t = \text{Model}(C_t)$$

where  $\hat{y}_t$  represents the predicted likelihood of each application being used next.

The system focuses on top- $k$  prediction, where the model returns the most probable candidate applications along with confidence scores.

### 3.7 Action Layer: Preloading Strategy

Based on the prediction output, the system performs user-level optimization through proactive preloading strategies. These include:

- Background initialization of predicted applications,
- Cache warming for application binaries and dependencies,
- Maintaining predicted applications in a standby state.

It is important to emphasize that the system operates entirely at the user level and does not modify the operating system kernel or scheduler. All optimizations are implemented through user-space mechanisms to ensure portability and safety.

To avoid unnecessary resource consumption, preloading is triggered only when the prediction confidence exceeds a predefined threshold. Additionally, the system can limit preloading to the top- $k$  predicted applications to balance performance gains and resource usage.

### 3.8 Cross-Platform Considerations

A key challenge in system design is the heterogeneity of tracing mechanisms across operating systems. Differences in observability, data formats, and available metrics require a unified abstraction layer that normalizes collected data into a consistent representation.

Despite these differences, the proposed architecture maintains portability by restricting itself to user-level operations and standardized feature representations. This design ensures that the core prediction and optimization components can be shared across platforms.

## 4 Modeling Approaches

### 4.1 Overview

The proposed system adopts a progressive modeling strategy that balances feasibility, predictive performance, and system overhead. Rather than committing to a single model, this study explores a spectrum of approaches with increasing complexity and capability.

The modeling framework is divided into three levels:

1. Statistical Transition Models (Baseline)
2. Sequence Learning Models (Core Approach)

### 3. Context-Aware Semantic Models (Advanced Extension)

This layered design enables systematic evaluation of trade-offs between simplicity, interpretability, and predictive power.

## 4.2 Level 1: Statistical Transition Models

### 4.2.1 Model Description

The most basic approach models user workflows as a stochastic process over application-level states. The probability of the next application is estimated based on transition frequencies observed in historical data.

$$P(a_{t+1} \mid a_t)$$

or, for higher-order dependencies:

$$P(a_{t+1} \mid a_{t-k}, \dots, a_t)$$

Transition probabilities are computed using empirical counts:

$$P(a_j \mid a_i) = \frac{\text{count}(a_i \rightarrow a_j)}{\sum_{a'} \text{count}(a_i \rightarrow a')}$$

### 4.2.2 Advantages

- Extremely simple to implement and interpret,
- Requires minimal training data,
- Computationally efficient with negligible runtime overhead,
- Provides a strong baseline for repetitive workflows.

### 4.2.3 Limitations

- Limited ability to capture long-term dependencies,
- No understanding of contextual or semantic information,
- Sensitive to sparse transitions,
- Poor generalization to unseen workflow patterns.

### 4.2.4 Role in the System

This model serves as a baseline for feasibility validation and performance comparison. It provides insight into whether simple statistical patterns are sufficient to capture user behavior.

## 4.3 Level 2: Sequence Learning Models

### 4.3.1 Model Description

To capture temporal dependencies beyond local transitions, the workflow is modeled as a sequence prediction problem. Given a context window:

$$C_t = (a_{t-k}, \dots, a_t)$$

the model predicts the next application:

$$P(a_{t+1} \mid C_t)$$

Candidate architectures include recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and lightweight Transformer models.

Each application is represented as an embedding vector, and the sequence is processed to produce a hidden representation encoding recent user behavior.

#### 4.3.2 Advantages

- Captures temporal dependencies across multiple steps,
- Adapts to complex and non-repetitive workflows,
- Supports incorporation of auxiliary features (time, resource usage),
- Provides a practical balance between accuracy and computational cost.

#### 4.3.3 Limitations

- Requires structured dataset construction,
- Less interpretable than statistical models,
- Increased training complexity,
- Still limited in understanding high-level user intent.

#### 4.3.4 Role in the System

This level constitutes the core predictive component of the system. It is expected to significantly outperform statistical models while remaining feasible for deployment on personal devices.

### 4.4 Level 3: Context-Aware Semantic Models

#### 4.4.1 Model Description

At the highest level, the system incorporates semantic understanding of user actions. Instead of modeling only application transitions, the system considers the underlying intent or task associated with each process.

This can be achieved by augmenting each event with semantic labels:

$$e_t = (a_t, s_t)$$

where  $s_t$  denotes a semantic tag such as *edit*, *compile*, *browse*, or *communicate*.

The prediction task becomes:

$$P(a_{t+1} \mid C_t, S_t)$$

where  $S_t$  represents the sequence of semantic states.

In more advanced configurations, a compact language model (e.g., ~0.8B parameters) may be introduced to infer contextual relationships and improve generalization.

#### 4.4.2 Advantages

- Captures high-level user intent rather than surface-level behavior,
- Improves generalization across different workflows,
- Enables reasoning over previously unseen sequences,
- Provides a pathway toward intelligent assistant systems.

#### 4.4.3 Limitations

- Requires reliable semantic labeling or inference,
- Increased system complexity and computational cost,
- More difficult to train and evaluate,
- Potential challenges in real-time deployment.

#### 4.4.4 Role in the System

This level is considered an advanced extension rather than a core requirement. It represents a long-term direction for enhancing predictive capability and adaptability.

### 4.5 Comparison of Modeling Approaches

| Model Type        | Complexity | Accuracy  | Feasibility |
|-------------------|------------|-----------|-------------|
| Statistical Model | Low        | Moderate  | High        |
| Sequence Model    | Moderate   | High      | High        |
| Semantic Model    | High       | Very High | Moderate    |

### 4.6 Design Strategy

The system follows a staged development approach:

- Phase 1: Implement statistical baseline for validation,
- Phase 2: Deploy sequence learning model as the primary predictor,
- Phase 3: Integrate semantic features and context-aware modeling.

This progression ensures that the system remains feasible at early stages while allowing incremental improvements in predictive performance and functionality.

## 5 Feasibility Analysis

### 5.1 Overview

This section evaluates the feasibility of the proposed system from the perspectives of data collection, system constraints, computational requirements, and deployment practicality. The analysis focuses on identifying both enabling factors and inherent limitations across supported platforms.

## 5.2 Data Collection Feasibility

The system relies on user-level tracing mechanisms available on modern operating systems:

- Linux: eBPF provides flexible and efficient kernel-level tracing with rich observability.
- macOS: dtrace enables process-level tracing but is partially restricted by System Integrity Protection (SIP).
- Windows: ETW (Event Tracing for Windows) provides structured event logging with lower flexibility compared to eBPF.

Despite differences in capabilities, all three platforms support monitoring of process creation and execution events, which is sufficient for constructing application-level workflows.

However, the level of detail varies:

- eBPF allows fine-grained instrumentation and custom probes,
- dtrace provides moderate observability with certain restrictions,
- ETW offers predefined event schemas with limited customization.

To address this heterogeneity, the system introduces a unified abstraction layer that normalizes collected data into a consistent format. Therefore, cross-platform data collection is considered feasible, though with varying levels of richness.

## 5.3 Process Abstraction Feasibility

A key challenge lies in distinguishing meaningful application-level processes from system-level noise. Modern applications often spawn multiple subprocesses, making direct process identification ambiguous.

This issue can be mitigated through:

- Process grouping based on parent-child relationships,
- Name-based filtering and whitelisting of target applications,
- Heuristic mapping from low-level processes to application identities.

While not perfectly accurate, these techniques are sufficient for capturing high-level workflow patterns, making process abstraction feasible in practice.

## 5.4 Model Training and Inference Feasibility

The proposed modeling approaches are computationally manageable on personal laptops:

- Statistical models require negligible computation,
- Sequence models (e.g., LSTM or small Transformer) can be trained on moderate datasets and support efficient inference,
- Advanced semantic models introduce higher overhead but remain feasible with compact architectures.

Inference latency is expected to be low, enabling real-time prediction. Since predictions are performed at discrete process events rather than continuously, the computational burden remains limited.

## 5.5 Preloading Strategy Feasibility

The system implements preloading entirely at the user level without modifying the operating system kernel. Feasible techniques include:

- Background launching of applications,
- Cache warming by accessing application binaries,
- Maintaining lightweight standby processes.

However, several constraints must be considered:

- Limited control over memory allocation and scheduling,
- Risk of increased resource consumption if predictions are inaccurate,
- Dependency on operating system policies for process management.

To mitigate these risks, preloading is conditioned on prediction confidence and limited to a small number of candidate applications.

## 5.6 Real-Time Constraints

The system operates in an event-driven manner, where predictions are triggered by process transitions. This design reduces computational overhead compared to continuous monitoring.

Key considerations include:

- Prediction latency must be negligible relative to application launch time,
- Preloading must not interfere with foreground tasks,
- Resource usage must remain within acceptable limits.

Given the lightweight nature of the models and event-driven design, real-time operation is feasible.

## 5.7 Single-User Initialization

The system is initially designed for a single user with a well-defined behavioral profile. This setup offers several advantages:

- Clean and consistent data collection,
- Faster convergence during model training,
- Easier validation of prediction accuracy.

However, this also introduces the risk of overfitting to individual behavior patterns.

## 5.8 Multi-User Generalization

Extending the system to multiple users introduces variability in workflows and application usage patterns. To address this, several strategies can be employed:

- User-specific models trained independently,
- Shared global model with user embeddings,
- Pretraining followed by user-level fine-tuning.

Among these, a shared model with user-specific adaptation provides a balance between scalability and personalization.

## 5.9 Privacy Considerations

Process-level data inherently reflects user behavior and may contain sensitive information. Therefore, the system must incorporate privacy-preserving mechanisms, including:

- Anonymization of collected data,
- Local storage and processing without external transmission,
- Optional user control over data collection scope.

Ensuring privacy is essential for practical deployment and user acceptance.

## 5.10 Overall Feasibility Assessment

The proposed system is feasible at the user level with currently available tools and computational resources. While certain limitations exist, particularly in cross-platform consistency and system-level control, these do not prevent the development of a functional and effective prototype.

The staged design approach further enhances feasibility by allowing incremental development and evaluation, starting from simple models and progressing toward more advanced capabilities.

# 6 Evaluation Plan

## 6.1 Evaluation Objectives

The evaluation is designed to assess the proposed system from two complementary perspectives:

- Predictive performance, namely whether the model can accurately anticipate the next useful application-level process,
- System effectiveness, namely whether accurate prediction can reduce application launch latency without introducing excessive overhead.

Accordingly, both machine learning metrics and system-level performance metrics are considered.

## 6.2 Prediction Metrics

The primary prediction task is top- $k$  next-application prediction. The following metrics are used:

- Top-1 Accuracy: the fraction of cases in which the most probable prediction matches the actual next application,
- Top- $k$  Accuracy: the fraction of cases in which the correct next application appears among the top- $k$  predictions,
- Precision and Recall: used when evaluating predictions over selected application categories,
- Confusion Matrix: used to analyze common misclassification patterns between applications.

These metrics allow comparison across statistical, sequence-based, and context-aware models.

### 6.3 System Metrics

To evaluate practical usefulness, the following system-level metrics are measured:

- Application launch latency reduction,
- CPU overhead introduced by tracing and inference,
- Memory overhead introduced by prediction and preloading,
- Preloading success rate, defined as the fraction of preload actions that are followed by actual user invocation,
- Wasted preload rate, defined as the fraction of preload actions that are not used within a specified time window.

These measurements determine whether improved predictive accuracy translates into meaningful system optimization.

### 6.4 Experimental Setup

The initial experiments focus on a single-user setting, where the user profile is well understood and application usage patterns are relatively stable. This setting supports controlled data collection and early-stage validation.

The applications of interest are restricted to frequently used work-oriented software, including development tools, document-processing tools, browsers, communication platforms, and productivity applications.

A typical workflow dataset is constructed from application-level process traces after filtering and abstraction. The dataset is then split into training, validation, and test segments according to chronological order to preserve temporal realism.

### 6.5 Experimental Plan

| Stage   | Data Scope                   | Model                     | Main Evaluation Target  |
|---------|------------------------------|---------------------------|-------------------------|
| Phase 1 | Single user, short-term logs | Statistical model         | Baseline accuracy       |
| Phase 2 | Single user, extended logs   | LSTM / small Transformer  | Prediction quality      |
| Phase 3 | Single user + semantic tags  | Context-aware model       | Intent-aware prediction |
| Phase 4 | Multiple users               | Personalized shared model | Generalization ability  |

### 6.6 Ablation Considerations

To understand the contribution of different design choices, the following comparisons are included:

- With and without time-based features,
- With and without resource-usage features,
- With and without semantic tags,
- Top-1 versus top- $k$  preloading strategies,
- Single-user model versus shared multi-user model.

These experiments help identify which components contribute most to predictive and system performance.

## 7 Key Challenges

### 7.1 Cross-Platform Inconsistency

Although Linux, Windows, and macOS all support process tracing, the available interfaces differ substantially. eBPF offers rich and flexible instrumentation, dtrace is subject to platform restrictions, and ETW provides more structured but less customizable event streams. These differences may affect data richness and model consistency across platforms.

### 7.2 Process Ambiguity

Many modern applications spawn multiple helper processes, renderer processes, or utility services. As a result, raw traces may not directly correspond to user-visible applications. Accurate grouping and abstraction are therefore necessary but may introduce heuristic error.

### 7.3 Semantic Labeling Difficulty

Advanced models benefit from semantic tags such as *edit*, *compile*, and *communicate*. However, such labels are not directly available from process traces and must be inferred from heuristics, metadata, or external context. This increases system complexity.

### 7.4 Prediction-Optimization Mismatch

A correct prediction does not always imply useful preloading. Some applications launch quickly and benefit little from preloading, whereas others consume significant memory and may introduce overhead if preloaded unnecessarily. Therefore, prediction quality and optimization benefit must be evaluated separately.

### 7.5 Personalization and Generalization

A model trained on a single user may achieve high performance but may not generalize well to others. Conversely, a shared multi-user model may lose user-specific accuracy. Balancing personalization and generalization is a central challenge for later-stage development.

## 8 Conclusion

This report has examined the feasibility of a cross-platform, user-level system for context-aware prediction of application-level processes and proactive application preloading. The proposed framework integrates process tracing, workflow modeling, predictive inference, and user-space optimization into a unified system designed for work-oriented personal computing environments.

The analysis suggests that the project is feasible under realistic constraints. Process-level data can be collected across Linux, Windows, and macOS using existing tracing tools. Application-level workflows can be extracted through filtering and abstraction. Predictive models ranging from simple statistical baselines to sequence learning architectures can be trained with manageable computational cost. Furthermore, user-level preloading strategies can be implemented without modifying the operating system kernel.

The most feasible development path begins with a single-user system, where workflow patterns are clearer and validation is easier. A statistical baseline provides an interpretable starting point, while sequence learning models offer the best balance between predictive performance and practical deployment. More advanced semantic models represent a promising extension when sufficient contextual information becomes available.

Overall, the project is both technically meaningful and practically achievable as a staged system study. Its value lies not only in predictive modeling, but also in the broader exploration of how process-level workflow understanding can support intelligent system optimization.

## 9 Future Work

Several directions may extend this work beyond the initial feasibility stage.

- Expanding from a single-user setting to multi-user datasets,
- Incorporating user embeddings for personalized adaptation,
- Improving semantic inference of workflow intent,
- Designing more refined preloading policies based on confidence and resource cost,
- Investigating lightweight assistant-style models for higher-level workflow reasoning,
- Evaluating long-term productivity gains in realistic user studies.

These extensions would further strengthen the connection between system tracing, user behavior modeling, and intelligent operating-system-level assistance.